

robust that we could deploy, debug, and maintain easily.

Here's a breakdown of what we used and why.

Frontend stack: We chose React 18.3 for the frontend mainly because of its component-based architecture and active ecosystem. Combined with TypeScript, it gave us the ability to catch type-related bugs early, which was especially helpful once the project grew beyond a dozen components.

To handle styling, we used:

- **Tailwind CSS:** Its utility-first approach helped us build responsive UIs quickly without writing long CSS files.
- **Lucide React:** For icons that were light and scalable enough to handle both the patient-facing and admin-facing views.

Other important tools used are:

- **React Router:** For managing routes and protected paths across user roles.
- **React Hot Toast:** Provided simple, non-intrusive notifications to improve feedback for users (like success or error messages).
- **React Context API:** We used it to store shared state like the logged-in user's role or current session without having to pull in a larger state manager like Redux.

Backend stack: The backend is built with Node.js and Express.js, which allows us to spin up services quickly with minimal boilerplate. We separated API routes into modules (e.g., auth, appointments, groups) and used middleware for token checks, request validation, and logging.

For real-time features, we added Socket.IO to manage WebSocket connections for updates like appointment status changes and support group activity.

Other utilities used are:

- **Axios** (used both frontend and backend): Helped standardise all HTTP calls.
- **Multer:** A reliable middleware for

handling file uploads—important for the resource-sharing feature.

Database

We went with MongoDB largely because of its flexibility. Healthcare data can be unpredictable — patients might upload files, doctors might add notes, and admins might tag resources with metadata. A rigid SQL schema would have slowed us down, especially during iteration.

Our main collections:

- Users
- Appointments
- Groups
- Resources

MongoDB also worked well with Node.js thanks to the Mongoose ODM, which simplified validation and querying.

Security utilities

Security was baked into the stack using:

- **JWT:** For stateless authentication. Tokens are issued at login and validated on every API call.
- **bcrypt:** For password hashing before storing credentials in the database.
- **Express Validator:** For input validation to prevent injection or malformed data.

Layer	Technologies used
Frontend	React 18.3, TypeScript, Tailwind CSS, Lucide React, React Router, React Context
Backend	Node.js, Express.js, Socket.IO, Axios, Multer
Database	MongoDB
Authentication	JWT, bcrypt, Express Validator

Table 1: Overview of core technologies used in SereneCare

Choosing the right tools saved us from getting stuck in debugging

rabbit holes and let us focus more on features. Almost everything we used has strong community support, which made troubleshooting much easier. Since SereneCare is open source, we also wanted it to be easy for others to pick up and contribute — and sticking with popular, well-documented libraries made that a lot more feasible.

Modular service design

As the feature list for SereneCare started expanding, we realised early on that trying to cram everything into a single backend file would quickly spiral into chaos. So instead of a monolithic structure, we broke the platform down into independent service modules, each one responsible for a specific domain like authentication, appointments, groups, or resource sharing.

This modular approach helped in two big ways:

- It kept the codebase manageable, especially when multiple people were working in parallel.
- It made debugging and scaling much easier, since each module could be developed, tested, or optimised on its own.

Role-based access and workflow mapping

At the heart of our design is a role-based access system. Each user logs in with a specific role: patient, doctor, or admin, and sees a tailored dashboard. Here's how we split the workflows.

- **Patients:** Can book appointments, join support groups, and browse shared resources.
- **Doctors:** Can manage consultations, publish resources, and moderate discussions.
- **Admins:** Can approve user accounts, monitor group activity, and oversee system-wide metrics.

We used JWT payloads to embed role information, so that every request could be easily verified against role-based permissions on the backend.